



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°5

Caché

Rodriguez del Busto, Sofia

- 1) ¿Cuántos bits en total se requieren para una caché de mapeo directo con 8 KB de datos y bloques de 8 palabras de 32 bits cada una, asumiendo una dirección de 64 bits? Detalle cuántos bits para el índice, la etiqueta, datos y validación.

Cada bloque tiene 8 palabras (2^3 , $m=3$) de 32 bits, luego cada bloque tiene un tamaño igual a $8 \times 32 = 256$ bits de datos.

El tamaño de la caché es 8 KB de datos, lo que equivale a 64 kb de datos.

Como cada bloque de caché es de 256 bits, la cantidad de bloques de la caché $64 \times 1024 \text{ bits} / 256 \text{ bits} = 256$ bloques.

Para el índice necesito una cantidad n de bits (la menor posible) tal que $2^n \geq 256$. Luego, $n = 8$ bits. Necesitamos un bit de validez.

El tamaño en bits de la etiqueta es igual a: $64 - (8 + 3 + 2) = 51$ bits

El número total de bits de la caché es: $2^n \times (\text{tamaño de bloque} + \text{tamaño de la etiqueta} + \text{campo de validación})$.

El número total de bits de la caché es: $2^8 \times (256 \text{ bits} + 51 \text{ bits} + 1 \text{ bit}) = 78848 \text{ bits}$.

- 2) ¿Qué significa que una memoria caché sea totalmente asociativa? Supongamos que tenemos una caché con 8 conjuntos y 4 líneas por conjunto. ¿Cómo se mapearían las direcciones de memoria en esta caché? ¿Cuál sería la función de búsqueda en esta estructura?

Una caché totalmente asociativa es aquella en la que cualquier bloque de memoria principal puede almacenarse en cualquier línea de la caché. No existen restricciones de ubicación como en el mapeo directo, donde cada bloque tiene un único lugar posible, o en el mapeo por conjuntos, donde solo puede ir a un conjunto específico.

Una caché de 8 conjuntos y 4 líneas por conjunto es una caché set associative donde cada bloque de memoria puede ubicarse en un único conjunto específico, pero dentro de ese conjunto puede ocupar cualquiera de las 4 vías.

Cada dirección de memoria se divide en tres campos: índice, que identifica el conjunto al que pertenece esa dirección, el tag que identifica al bloque de memoria en el conjunto y el campo offset, que indica su posición dentro del bloque.

Para mapearla, se debe identificar el número de conjunto al que pertenece. Esto se obtiene como $\text{número de conjunto} = (\text{número de bloque en memoria}) \bmod (\text{cantidad de conjuntos})$. En este caso, por ejemplo, el bloque 26 pertenecería al segundo conjunto ya que $26 \bmod 8 = 2$. El bloque dentro del conjunto puede ocupar cualquiera de las líneas disponibles.

Para buscar dentro de esta estructura de caché, se identifica el índice de la dirección que indica el conjunto al que pertenece. Identificado el conjunto se busca dentro de él. Dentro del conjunto, se compara el tag de la dirección con los tags de cada una de las vías de ese conjunto. Si existe una coincidencia se produce un HIT y el dato se toma de la línea. En caso de que se produzca un MISS, el



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°5

Caché

Rodriguez del Busto, Sofia

bloque se trae de memoria principal y ocupa o reemplaza una de las vías de la caché. En caso de reemplazo se puede usar alguna política como LRU (Least Recently Used).

- 3) Una computadora de 32 bits con una memoria caché de 256 KB y líneas de 64 bytes. La caché es asociativa por conjuntos de 4 vías. Se pide:

- a) Indique el número de líneas y de conjuntos de la memoria caché del enunciado.

El número de líneas de caché = tamaño de la caché/tamaño de líneas de caché = $256 * 1024 \text{ B} / 64 \text{ B} = 4096$ líneas de caché.

Si por cada conjunto tenemos 4 vías de caché y tenemos 4096 líneas de caché, la cantidad de conjuntos es igual a: cantidad de líneas de caché/cantidad de vías = $4096 / 4 = 1024$ conjuntos.

- b) ¿Cuál es el tamaño de los bloques que se transfieren entre la memoria caché y la memoria principal?

El bloque transferido entre memoria principal y caché es una línea de caché, es decir, 64 bytes.

- 4) Se dispone de una computadora con direcciones de memoria de 32 bits, que direcciona la memoria por bytes. La computadora dispone de una memoria caché asociativa por conjuntos de 4 vías, con un tamaño de línea de 4 palabras. Dicha caché tiene un tamaño de 64 KB. Indique de forma razonada:

- a) Tamaño en MB de la memoria que se puede direccionar en este computador.

Si tenemos direcciones de memoria de 32 bits y se direcciona la memoria por bytes, entonces podemos direccionar 2^{32} bytes. Esto es equivalente a $2^{12} * 2^{20} \text{ Bytes} = 4096 \text{ MB}$.

- b) Número de palabras que se pueden almacenar en la memoria caché.

Si mi caché tiene un tamaño de 64 KB y suponemos que el tamaño de palabra es de 4 bytes por palabra, la cantidad de palabras que puedo almacenar en caché es: $64 * 1024 \text{ bytes} / 4 \text{ bytes/palabra} = 16384$ palabras

- c) Número de líneas de la caché.

Si mi línea de caché tiene 4 palabras y cada una de ellas es de 4 bytes, la cantidad de bytes por línea de caché es $4 * 4 \text{ bytes} = 16 \text{ bytes}$.

Sabemos que el tamaño de la caché es de 64 KB. Luego, el número de líneas de caché será igual a:

Tamaño de la caché/tamaño de línea de caché = $64 * 1024 \text{ bytes} / 16 \text{ bytes} = 4096$ líneas de caché.

- d) Número de conjuntos de la caché.

La cantidad de líneas de caché es 4096 por lo calculado anteriormente y como la caché es de 4 vías cada conjunto contiene 4 líneas de caché. Por lo tanto, el número de conjuntos de la caché será igual a: $n^\circ \text{ de líneas de caché} / n^\circ \text{ de vías por conjunto} = 4096 / 4 = 1024$ conjuntos.



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°5

Caché

Rodriguez del Busto, Sofia

5) De acuerdo a un estudio realizado sobre la utilización de las instrucciones de una computadora RISC, en promedio ejecuta 50 millones de instrucciones por segundo y que el porcentaje de utilización de sus instrucciones es el siguiente:

- LOAD un 30 %
- STORE un 15 %
- Operaciones aritméticas un 29 %
- Operaciones lógicas un 6 %
- Saltos un 20 %

a) Determine el número de accesos a memoria por segundo que se realizan en esta computadora.

Por cada operación LOAD y STORE hay un acceso a memoria. El resto de las instrucciones no realizan accesos a memoria ya que operan con registros del procesador. Por lo tanto, la cantidad de accesos a memoria por dichas operaciones se calcula como:

Porcentaje de utilización de LOAD * promedio total de instrucciones + Porcentaje de utilización de STORE * promedio total de instrucciones = $0,30 * 50 * 10^6$ instrucciones/s + $0,15 * 50 * 10^6$ instrucciones = $22,5 * 10^6$ instrucciones/s.

b) BONUS: ¿Cuál será el número de accesos a memoria principal por segundo en caso de utilizar una memoria caché con un tamaño de línea de 8 palabras, política de actualización write-through (escritura inmediata) y una tasa de aciertos del 95 %?

6) Basados en el ejemplo de la teoría, determinar en hay Hit o Miss en cada referencia de la secuencia y completar el bloque de caché desde su estado inicial.

Como tengo 8 líneas de caché necesito $n=3$ bits para el índice ya que me permite representar $2^3 = 8$ índices. Por lo tanto, las siguientes direcciones de caché se asignarán al bloque cuyo índice coincida con los 3 bits menos significativos de la referencia de dirección.

Referencia de dirección (decimal)	Referencia de dirección (binario)	Hit o Miss en Caché	Bloque de cache asignado
3	$(00011)_2$	Miss(B)	$(011)_2$
3	$(00011)_2$	Hit	$(011)_2$
21	$(10101)_2$	Miss(C)	$(101)_2$
7	$(00111)_2$	Miss(D)	$(111)_2$
21	$(10101)_2$	Hit	$(101)_2$
27	$(11011)_2$	Miss(E)	$(011)_2$
5	$(00101)_2$	Miss(F)	$(101)_2$



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°5

Caché

Rodriguez del Busto, Sofia

7	$(00111)_2$	Hit	$(111)_2$
24	$(11000)_2$	Miss(G)	$(000)_2$

Estado inicial de la caché

Índice		V	Tag	Data
000	N			
001	N			
010	N			
011	N			
100	N			
101	N			
110	N			
111	N			

B) Después del miss de la dirección $(00011)_2$

Índice		V	Tag	Data
000	N			
001	N			
010	N			
011	S	00_2		Memoria $(00011)_2$
100	N			
101	N			
110	N			
111	N			

C) Después del miss de la dirección $(10101)_2$

Índice		V	Tag	Data
000	N			



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II

TRABAJO PRÁCTICO N°5

Caché

Rodriguez del Busto, Sofia

001	N		
010	N		
011	S	00_2	Memoria(00011_2)
100	N		
101	S	10_2	Memoria(10101_2)
110	N		
111	N		

D) Después del miss de la dirección (00111_2)

Índice	V	Tag	Data
000	N		
001	N		
010	N		
011	S	00_2	Memoria(00011_2)
100	N		
101	S	10_2	Memoria(10101_2)
110	N		
111	S	00_2	Memoria(00111_2)

E) Después del miss de la dirección (11011_2)

Índice	V	Tag	Data
000	N		
001	N		
010	N		
011	S	11_2	Memoria(11011_2)
100	N		
101	S	10_2	Memoria(10101_2)



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II

TRABAJO PRÁCTICO N°5

Caché

Rodriguez del Busto, Sofia

110	N		
111	S	00 ₂	Memoria(00111) ₂

F) Después del miss de la dirección (00101)₂

Índice	V	Tag	Data
000	N		
001	N		
010	N		
011	S	11 ₂	Memoria(11011) ₂
100	N		
101	S	00 ₂	Memoria(00101) ₂
110	N		
111	S	00 ₂	Memoria(00111) ₂

G) Después del miss de la dirección (11000)₂

Índice	V	Tag	Data
000	S	11 ₂	Memoria(11000) ₂
001	N		
010	N		
011	S	11 ₂	Memoria(11011) ₂
100	N		
101	S	00 ₂	Memoria(00101) ₂
110	N		
111	S	00 ₂	Memoria(00111) ₂

7) Describe la política de reemplazo LRU (Least Recently Used) y cómo se utiliza en la memoria caché.

La política de reemplazo LRU (Least Recently Used) se utiliza en memoria caché asociativa para decidir
 M. Sc. Ing. Ticiano J. Torres Peralta 04/11/2025
 Ing. Pablo G. Toledo



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°5

Caché

Rodriguez del Busto, Sofia

qué línea de caché se reemplazará dentro de un conjunto en caso de que se produzca un fallo de caché y se necesita traer un bloque de memoria principal. Esta política reemplaza la línea que hace más tiempo no se usa. Es decir, aplica el principio de localidad temporal. Supone que los datos o instrucciones recientemente usados tiene una mayor probabilidad de ser utilizados en la brevedad. Este algoritmo mantiene una lista ordenada de cada entrada. Cada vez que se accede a cualquiera de las líneas presentes, actualiza la lista, marcando esa entrada como la entrada a la que se accedió más recientemente. Cuando llega el momento de reemplazar una entrada, la que está al final de la lista (la a la que se accedió menos recientemente) es la que se descarta.

- 8) Explica las diferencias entre la escritura en caché por escritura inmediata(write-through) y escritura en caché por escritura diferida (write-back). ¿Cuáles son las ventajas y desventajas de cada enfoque de escritura en caché?

En la escritura inmediata (write-through), cada vez que se modifica un dato en la caché, se realiza la escritura inmediatamente en la memoria principal. De esta manera, se garantiza que la memoria principal esté siempre actualizada con respecto a la caché. Esto puede resultar ventajoso para mantener la consistencia de los datos, es decir, que la memoria principal tenga siempre la versión más actualizada de los datos. Además, requiere de una implementación muy simple. Sin embargo, los continuos accesos a memoria principal disminuirán el rendimiento del sistema ya que es significativamente más lenta que la caché. También, se debe tener en cuenta que aumenta el tráfico en el bus de memoria al propagar cada modificación de la caché a la memoria principal.

En el caso de la escritura diferida (write-back), cuando se modifica un bloque de caché esta actualización se realiza solo en la caché. Recién se actualiza la memoria principal cuando el bloque es reemplazado en la caché. Para ello, se marca a los bloques cuyo contenido difiere del contenido de memoria principal de manera que, cuando salga de caché, se escriba el valor en memoria principal. Este método provee un mejor rendimiento ya que muchas de las escrituras se realizan únicamente en la caché y ayuda a disminuir el tráfico en el bus de memoria. No obstante, se necesita de una implementación más compleja para poder gestionar los bloques diferidos y controlar la sincronización. Otra desventaja es que pueden generarse inconsistencia si el sistema falla cuando aún bloque no se ha actualizado aún en memoria principal.

- 9) Describe los conceptos de localidad temporal y localidad espacial en el contexto de la memoria caché. Explica cómo la localidad puede afectar el rendimiento de la memoria caché.

La localidad se refiere al patrón predecible con el que los programas acceden a la memoria. Los sistemas de memoria caché aprovechan esta propiedad para aumentar la probabilidad de que los datos solicitados por el procesador ya estén en la caché, reduciendo los accesos a memoria principal.

La localidad temporal se refiere a la tendencia de un programa a reutilizar los mismos datos o instrucciones en un corto período de tiempo. Esto significa que, si un valor o instrucción fue accedido recientemente, es muy probable que se vuelva a acceder a él en la brevedad. Por ejemplo, ocurre con variables que se usan dentro de bucles, contadores o datos temporales que se actualizan varias veces. La localidad espacial se basa en la tendencia de los programas a acceder a posiciones de memoria que se encuentran próximas entre sí. Es decir, cuando se accede a una dirección de memoria, es muy



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°5

Caché

Rodriguez del Busto, Sofia

probable que pronto se necesiten datos almacenados en direcciones cercanas. Esto sucede, por ejemplo, al recorrer un arreglo o al ejecutar instrucciones de manera secuencial en un programa. Para aprovechar esta propiedad, la memoria caché carga la dirección solicitada y las adyacentes dentro de un mismo bloque para evitar que futuros accesos a datos requieran de acceder a memoria principal.

El aprovechamiento de los patrones de acceso a los datos permite mantener en la caché la información más relevante y anticipar futuras necesidades del procesador. Esto se traduce en menos fallas de caché y menos accesos a la memoria principal para mejorar el rendimiento del sistema.